主脚本说明（main.py）

一、代码整体作用说明

该脚本是 **单个输出维度（output dimension）** 的完整流水线入口，主要用于完成以下任务：

- 训练 MLP 模型
- 导出训练完成的模型为 **ONNX** 格式
- 对最终模型进行评估
- 基于模型生成预测结果（`gen`）

命令行参数控制项

脚本通过命令行参数灵活控制整体流程，主要包括：

- 当前处理的 **输出维度（output dimension）**
- 数据类型：
  - `Stress`
  - `Strain`
  - `Dis`
- 是否执行 **训练阶段**

设计目标

该脚本的核心设计目标是：

> **通过一条命令完成「训练 → 导出 → 评估 → 推理数据生成」的完整流程。**

二、整体执行流程（高层逻辑）

流程说明

- **main()**
  - 作为入口函数，负责解析命令行参数并调度各功能模块
- **train()**
  - 根据参数决定是否执行模型训练
  - 训练完成后导出 ONNX 模型
- **eval()**
  - 对当前输出维度下的最优模型进行性能评估
- **gen()**
  - 使用最终模型生成推理结果，用于后续分析或应用

三、train() 函数说明（核心配置集中区）

3.1 主要作用

`train()` 是**训练与模型导出阶段的统一封装函数**，承担了整个流程中最核心的配置与调度职责，主要包括：

- 定义并集中管理**所有模型超参数**
- 构造并规范化**模型保存目录结构**
- 根据 `with_train` 参数决定是否执行训练过程
- **无论是否训练，始终执行 ONNX 模型导出**

该函数是模型实验可复现性和流程一致性的关键。

3.2 关键超参数说明（维护时最常用）

参数名	含义	说明
`output_dim`	输出维度索引	每次只训练一个输出维度
`hidden_dim`	隐藏层维度	决定网络容量，影响精度与推理速度
`block_num`	网络 block 数	控制模型深度
`epoch_num`	训练轮数	默认 25
`batchsize`	训练 batch size	与显存占用直接相关
`lr / lr_min`	学习率范围	与 cosine scheduler 配合使用
`use_cosine`	是否使用 cosine 学习率	默认开启
`use_dropout / dropout_p`	Dropout 配置	当前默认关闭
`use_L2 / L2weight`	L2 正则化	当前默认关闭
`use_standerscale`	是否使用标准化	必须与推理阶段保持一致

注意事项：

- `use_standerscale` 会直接影响：
  - 训练阶段输入
  - 评估阶段输入
  - 推理阶段输入
- 三个阶段的该参数**必须完全一致**
- 不建议在已有模型基础上随意修改该配置，否则结果将不可比或失效

3.3 模型保存路径设计（非常重要）

模型保存路径采用**参数编码式命名**方式，其构造示例如下：

```
folder_name = (
    models/{name_psd}/{Stype}/
    output_dim{...}_hdim{...}_block{...}_epoch{...}_lr{...}_...
)
```

设计说明

- 所有训练配置都会被编码进路径名
- 方便不同实验并存，并可快速回溯模型来源
- **不建议手动修改路径拼接规则**
  否则在评估与推理阶段可能无法正确定位模型文件

3.4 with_train 的含义

```
if with_train:
    train_model.train(...)
```

- `with_train = True`
  执行训练 + 导出
- `with_train = False`
  跳过训练，直接使用已有模型导出 ONNX

适用场景：

- 已有模型，仅重新导出或重新推理

四、eval() 函数说明（模型性能评估）

4.1 主要作用

`eval()` 用于对训练完成后的模型进行性能评估，主要包括：

- 在指定模型目录下选择目标模型
- 对最终模型进行评估（误差、精度等指标）

当前评估逻辑说明：

- 仅评估最终模型（如 `mlp_model_best.pth`）
- 历史 epoch 模型的评估代码已保留，但默认未启用

4.2 关键参数说明

参数名	作用
`folder_name`	模型所在目录
`model_name`	指定评估的模型文件
`use_standerscale`	输入数据是否进行标准化
`outputdim`	输出维度
`Stype`	数据类型
`basePath`	原始数据路径

注意事项：

- `use_standerscale` **必须与训练阶段保持一致**
- 否则评估输入分布将发生偏移，评估结果将不具备任何参考意义

五、main() 函数说明（脚本入口）

5.1 命令行参数

参数	含义
`--i`	输出维度索引（默认 0）
`--stype`	数据类型（Stress / Strain / Dis）
`--with_train`	是否执行训练

使用示例：

```
for i in {0..8}; do nohup python3 main.py --i $i --stype Dis --with_train
> train$i.log 2>&1 & done
```

5.2 数据集与工程约定

```
name_psd = "gear19_2gearContact_geneData_Sp"
basePath = f"dataset/{name_psd}"
```

- `name_psd` 决定数据集名称
- 所有训练、评估与推理流程共享同一个 `basePath`

5.3 最终三步执行流程

```
train() → eval() → gen()
```

- `train()`：训练模型并导出 ONNX
- `eval()`：评估模型精度
- `gen()`：生成最终推理数据（供后续 C++ / ONNX 推理使用）

六、注意事项

- 不要随意修改超参数命名规则
  模型路径与加载逻辑强依赖这些信息

- `use_standerscale` 必须在全流程中保持一致
  包括：训练 / `eval` / `gen` / C++ 推理
- 脚本一次只处理一个 `output_dim`
  多维训练通过后台多进程或外部循环完成
- `with_train = False` 忽略训练部分
  仍然会导出 ONNX，并继续执行 `eval()` 与 `gen()`

作者： 雷尚谕
日期： 2025-12